Resume Screening with NLP

Author Name(s): Aryan Agarwal
Affiliation(s): School of Computer Science, UPES
Email(s): aryanagarwal.dev@gmail.com

Abstract

Recruiters have to deal with an enormous amount of resumes for every vacancy announcement, which makes manual screening slow and prone to errors. In this project, the team develops an automated resume screening system that uses Natural Language Processing (NLP) and machine learning to classify resumes into job roles, rank them against a job description by match score, and produce concise AI summaries. We preprocess resume texts (cleaning, stopword removal, lemmatization), extract features using TF-IDF (unigrams and bigrams), and train a One-vs-Rest logistic regression classifier. Model evaluation: k-fold cross validation, validation set, test set. For the HR use cases, we compute cosine-similarity between cleaned job description and candidate resumes to generate a match score, and produce short two-line summaries using Google Gemini. System saves the trained models (role_classifier_ovr_lr.joblib, tfidf_vectorizer.joblib), and the system outputs ranked results, downloadable CSV and PDF reports. Results printed by the notebook are competitive in terms of classification performance and highly consistent.

1. Introduction

With each job posting, organizations typically receive hundreds and even thousands of applications. Automation in resume screening has become essential. Manual screening is slow, inconsistent, and susceptible to unconscious bias. By applying NLP and ML, resumes can be objectively analyzed for relevant skills, experience, and contextual information that enables a system to classify resumes automatically, match them against job requirements, and rank candidates based on relevance to a specific job description. The result is faster, fairer, and more efficient recruitment.

1.1 Problem Statement

The manual resume screening process is not scalable, causes significant delays, and can produce inconsistent candidate shortlists. The goal is to create an automated pipeline that classifies resumes into job roles, ranks them by relevance to a job description, and produces short summaries to help recruiters decide efficiently.

1.2 Research Gap

Most of the existing resume screening systems either rely on simple keyword matching, which is unable to capture context, synonyms, or semantic meaning, while more advanced deep learning approaches require very large labeled datasets, an immense volume of training resources, and significant complexity in deployment, thus being unfit for small or medium-sized organizations. In contrast, there is a lack of lightweight models that are accurate and easy to deploy for resume classification and matching. We address this gap by employing a mid-complexity approach based on TF-IDF vectorization and Logistic Regression in this work, which demonstrates strong performance even with modest dataset sizes and has advantages regarding explainability, efficiency, and ease of implementation.

1.3 Objectives

- To build a machine learning model that can automatically classify and rank resumes according to job descriptions.
- To use a labeled resume dataset for model training and evaluation.
- To apply NLP methods such as TF-IDF and text embeddings to extract meaningful features from resumes.
- To create a working demo that shows how NLP can help in real-life recruitment.
- To compute a match score for each resume, indicating how closely it aligns with the required job role.
- To generate AI-based resume summaries to help recruiters quickly understand key candidate information.

2. Literature Review

The earlier resume screening tools relied heavily on keyword matching, which often resulted in poor performance as it could not understand context or meaning. Later on, TF-IDF, coupled with machine learning models like Logistic Regression and SVM, were employed by various researchers, improving accuracy even on small datasets. Deep learning models like BERT work well, but they demand large volumes of training data and are challenging for normal organizations to deploy. Some studies also utilized cosine similarity to find the best match between the resume and the job description. Despite so many attempts, what is still needed is a simple, fast, and practical system that yields good performance without huge datasets. This project bridges that gap by employing TF-IDF, Logistic Regression, cosine similarity, and AI summarization techniques to propose an efficient solution for resume screening.

3. Proposed Methodology

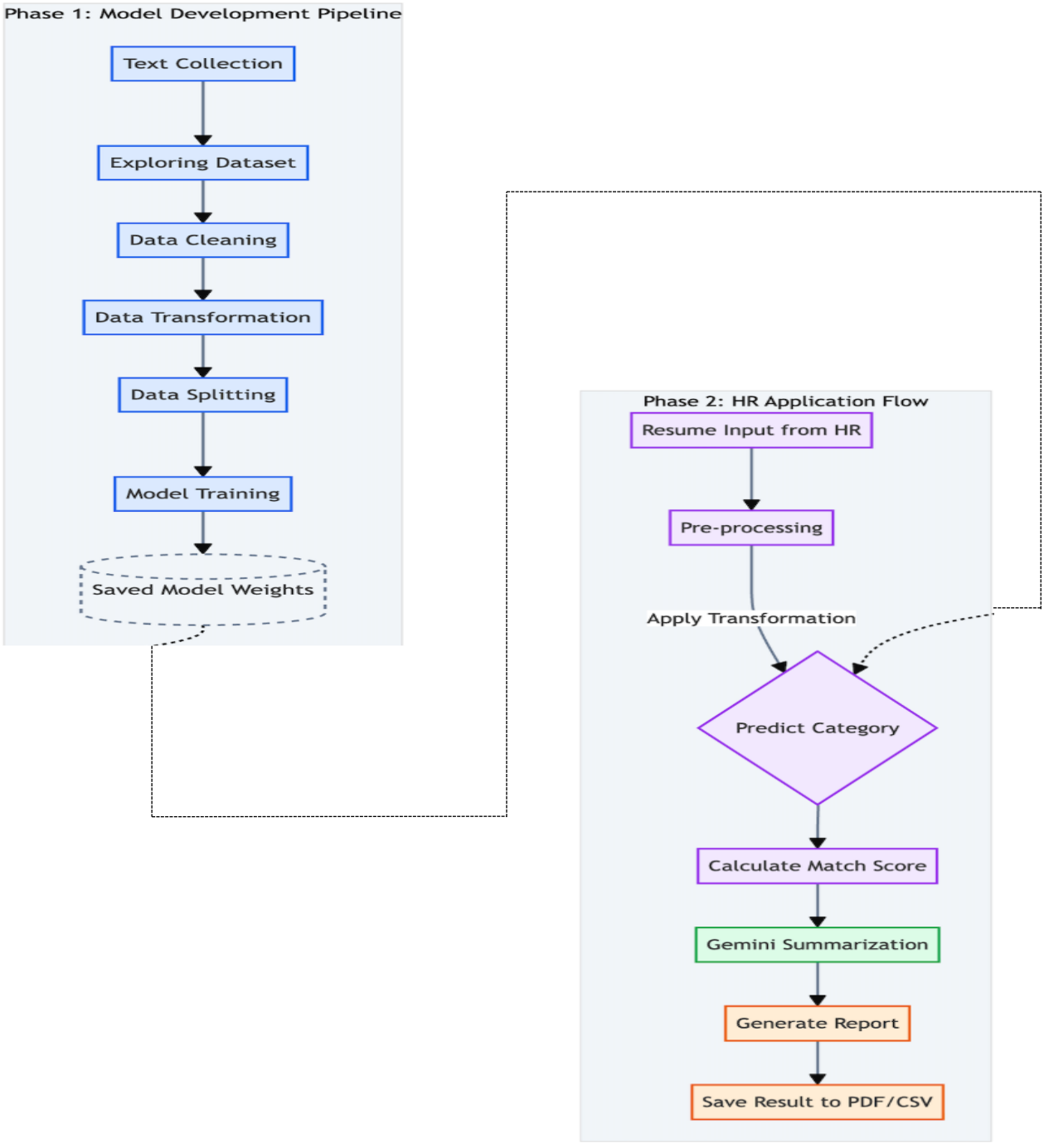

Fig. 1: Flowchart of our Model

Phase 1: Training

1. **Text Collection:** Gathering raw resume data and job descriptions.
2. **Exploration & Cleaning:** Analyzing distribution and removing noise (HTML tags, stop words).
3. **Transformation:** Converting text to vectors (TF-IDF, Word2Vec, BERT).
4. **Splitting & Training:** separating data into train/test sets and fitting the classifier.

Phase 2: Inference (HR Usage)

1. **Input:** HR uploads a specific candidate's resume.
2. **Prediction:** The saved model predicts the job category (e.g., "Java Developer").
3. **Scoring:** Calculates fit based on keyword overlap or cosine similarity.
4. **Gemini AI:** Generates a human-readable summary of strengths/weaknesses.
5. **Output:** Data is exported for the HR system.

3.1 System Model

The system model describes a modular pipeline where resumes are collected, cleaned using NLP, converted into TF-IDF features, classified using logistic regression, ranked using cosine similarity, summarized using Gemini, and exported as structured PDF/CSV reports.

3.2 Data Preprocessing

Preprocessing is a necessary step to transform the raw text of the resume into a structured, meaningful, and machine-readable format. The preprocessing pipeline of the project follows a sequential approach. It cleanses the data first, analyzes it with visualization, and then transforms it for feature extraction.

3.2.1 Data Collection (Dataset)

The dataset contains 10,174 labeled resumes, where each record includes the complete resume text along with its corresponding Job Category / Role.

This dataset is specifically curated for AI-based resume classification and match-score prediction tasks.

Dataset Source:

- Dataset Name: Resume-Screening-Dataset
- Platform: *Hugging Face*
- Uploader: AzharAli05
- Local File Used: Dataset.csv
- Total Rows: 10,174 resume entries
- Attributes: Role, Resume, Decision, Reason_for_decision, Job_Description

3.2.2 Data Cleaning

The first step was to clean the resume text to eliminate irrelevant noise that might interfere with analysis. This entailed converting all text to lower case, removing punctuation, special characters, numbers, and extra white space to yield cleaner and more consistent text that could be further processed.

- Convert all data to string format.
- Remove AI-generated intro lines.
- Convert text to lowercase.
- Remove emails, domain names, URLs, and phone numbers.
- Remove special characters and extra whitespace.
- Remove standalone numbers.
- Remove stopwords.
- Apply lemmatization to get root words.
- Standardize job roles and preserve acronyms (UI, UX, AI, HR).

3.2.3 Data Visualization (After Cleaning)

After the text was cleaned, visual plots for category distribution and common skills were created to understand the dataset. These visuals illuminated class imbalance and key patterns that helped guide further preprocessing and model design.

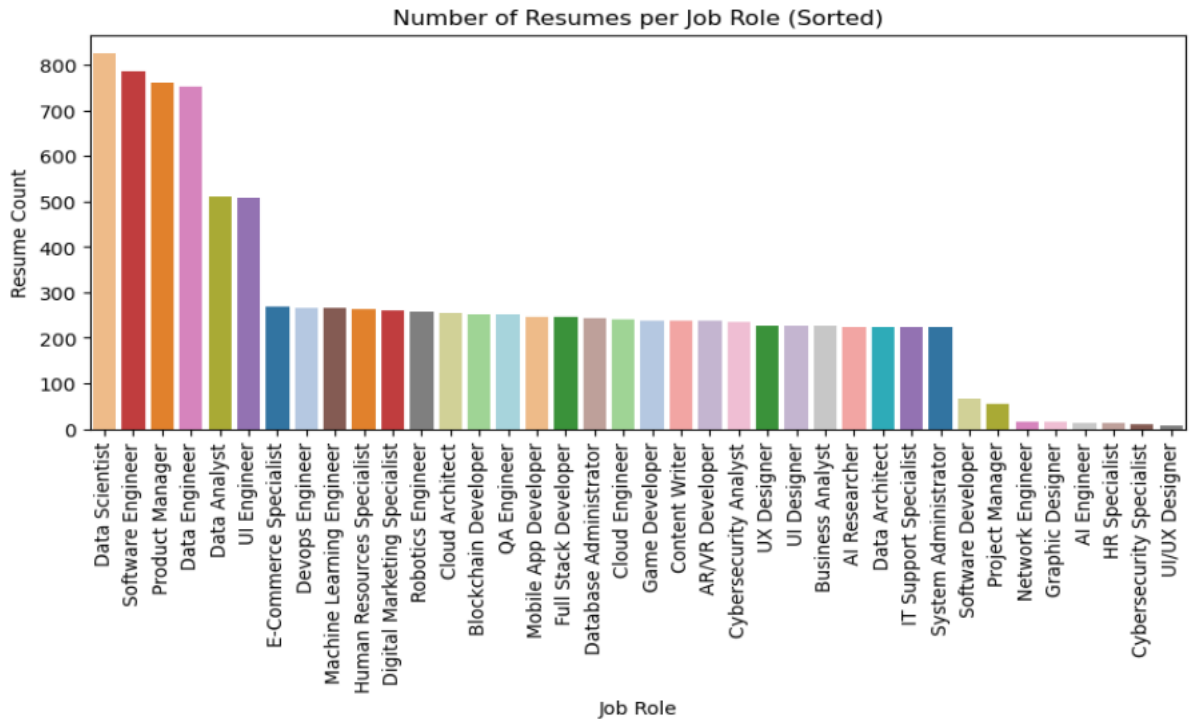

Fig. 2: Bar Chart
It is a bar chart showing the distribution of resumes across various job roles in the dataset. This helps in visualizing class imbalance, showing which roles have a high number of samples and which have few, which can be very important during model training and evaluation.

3.2.4 Data Transformation (Feature Extraction (TF-IDF))

The last step in the preprocessing pipeline is a conversion of text into numerical features using TF-IDF. This representation takes into consideration both word frequency and its importance across resumes.

The configuration used:
max_features = 20,000
n-gram range = (1,2)

The resulting TF-IDF vectors are used as input for the machine learning classifier.

```
tfidf = TfidfVectorizer(max_features=20000, ngram_range=(1,2))
X = tfidf.fit_transform(df["Cleaned_Resume"])
y = df["Role"].astype(str)

print("TF-IDF vectorization complete!")
print("Shape of feature matrix:", X.shape)
```

[83] Python

```
TF-IDF vectorization complete!
Shape of feature matrix: (10174, 20000)
```

Fig. 3: TF-IDF Vectorization of Cleaned Resume Text

3.2.5 Data Splitting

The dataset is divided into:
70% Training
15% Validation
15% Testing

Fig. 4: Dataset Splitting into Training, Validation, and Testing Sets

3.3 Algorithm / Framework

The proposed system uses a TF-IDF based text classification framework to identify the most suitable job role for a given resume. The textual data is first transformed into numerical representations using TF-IDF vectorization with a maximum of 20,000 features and an n-gram range of 1–2, enabling the model to capture both individual words and meaningful word pairs. These vectors are then fed into a One-vs-Rest Logistic Regression classifier configured with the ‘saga’ solver and 3000 maximum iterations to ensure efficient convergence on high-dimensional sparse data. To evaluate the model’s robustness and generalization capability, 5-Fold Cross-Validation is employed, providing a reliable estimate of performance across multiple data splits. This combined framework offers an efficient and scalable solution for multi-class resume categorization.

3.4 Model Training

The model is developed using a One-vs-Rest Logistic Regression framework. A classifier with a base estimator of Logistic Regression, using the 'saga' solver and a maximum number of iterations of 3000, is applied as an estimator for the sake of handling high-dimensional data efficiently. Subsequently, the stability and performance of this model are verified by carrying out 5-Fold Cross-Validation via the K-Fold strategy, including shuffling for randomness. For measuring generalization, it trains and validates on five different splits of the training dataset, then calculates the average accuracy score. After the process of cross-validation, the final model is trained on the full training set in order to learn the patterns associated with each category of job role. Once the training is complete, this trained model will be used for the prediction of job roles of new resumes.

Fig. 5: Model Training and 5-Fold Cross-Validation Procedure

3.5 Mathematical Formulation

TF-IDF Weighting: TF-IDF assigns weights to words based on how important they are in a document compared to the whole dataset.

Formula: TFIDF(t,d) = TF(t,d) * log (N / DF(t))

where,

- **TF(t, d)** = Term Frequency (how many times word *t* appears in document *d*)
- **DF(t)** = Document Frequency (number of documents that contain word *t*)
- **N** = Total number of documents in the dataset

Cosine similarity: Cosine similarity then compares the TF-IDF vectors of the job description and resume to compute a relevance score between 0 and 1.

Formula: sim (J, R) = (J · R) / (||J|| * ||R||)

where,

- J·R = Dot product of the two vectors
- ‖J‖ and ‖R‖ = Magnitudes (lengths) of the vectors

4. Experimental Setup

To execute the resume classification and summarization pipeline, a standard Python-based data science environment was utilized. The system was implemented using the following hardware and software configurations:

Hardware Configuration:

- Processor: **M3 Chip**
- RAM: **8 GB**
- Storage: 512**GB SSD**
- Operating System: **macOS**

Software & Libraries:

- **Programming Language:** Python (3.12.7)
- **Development Environment:** Jupyter Notebook / VS Code / Google Colab
- **Key Python Libraries Used:**
  - **pandas** – Data handling and manipulation
  - **numpy** – Numerical operations
  - **scikit-learn** – Model building and evaluation
  - **nltk / re** – Text preprocessing and tokenization
  - **joblib** – Model saving and loading
  - **PyPDF2** – Resume text extraction from PDFs
  - **google-generativeai** *(optional)* – For Gemini-based summarization
  - **reportlab** – Generate final PDF reports

Version Control & Deployment:

- Google Colab
- Git/GitHub

5. Results and Discussion:

Test Name	Accuracy	Precision	Recall	F1 Score	Definition
K-Fold Cross Validation (5-Fold)	0.9935 (Mean)	-	-	-	Evaluates model stability across 5 different train/test splits using only training data.
Validation Performance	**0.9934**	**0.9891**	**0.9934**	**0.9911**	Measures performance on the validation set (unseen during training).
Final Test Performance (Unseen Data)	**0.9954**	**0.9929**	**0.9954**	**0.9941**	Measures performance on completely unseen data which is the real estimate of model accuracy.

Interpretation (Class Imbalance): The high cross-validation and test accuracies indicate strong model performance on this dataset. However, note the UndefinedMetricWarning printed by sklearn: some rare classes had zero predicted samples leading to undefined precision for those labels. While weighted averages remain high, care should be taken with very small classes (e.g., 'AI Engineer' with only a couple of samples).

6. Conclusion

This project proposes a practical and effective resume screening system which can classify, score the relevance of candidates, and summarize their profiles automatically. By coupling TF-IDF vectorization with a One-vs-Rest Logistic Regression classifier, the system achieves high accuracy, fast inference, and strong performance even on modestly sized datasets. The pipeline also incorporates cosine similarity for job-resume matching and Gemini-based summarization to provide clear, concise insights to HR teams.

The model is lightweight, interpretable, and straightforward to deploy, making it suitable for small and medium-sized organizations that may not have available large datasets or heavy computational resources. Experimental results confirm strong generalization; however, caution must be exercised in the case of very small role categories, where model predictions are less reliable.

Future enhancements could be OCR support for scanned PDF resumes, integration of deep contextual embeddings, such as BERT or Sentence Transformers, expansion of the dataset to include more job roles, handling of imbalanced classes through resampling techniques, and deployment as a secure Streamlit or web-based interface for real-time HR use. With these improvements, the system can evolve into a robust and scalable ATS solution.

7. References

- IBM – Natural Language Processing. https://www.ibm.com/think/topics/natural-language-processing
- GeeksforGeeks – NLP Overview. https://www.geeksforgeeks.org/nlp/natural-language-processing-overview
- Ali, Azhar. Resume-Screening-Dataset. Hugging Face. https://huggingface.co/datasets/AzharAli05/Resume-Screening-Dataset/viewer
- HarsH-hP. Resume Screening NLP Project (GitHub). https://github.com/HarsH-hP/Resume-Screening-NLP-Project
- YouTube tutorial: Resume Screening using NLP and Machine Learning. https://youtu.be/hG8K5h2J-5g?si=MpdV7_9fccoEIrsc
- Online Available Model - ResumeScreening.ai. https://resumescreening.ai/
- Online Available Model - ResumeWorded. https://resumeworded.com/resume-scanner